

Análisis de los problemas del desbalanceo de clases en Predictive Mutation Testing

INTRODUCTION

El **software testing** es una fase crucial en el ciclo de vida de desarrollo de software y tiene como objetivo identificar y eliminar fallos para asegurar la alta calidad de los productos software.

El **mutation testing** es una técnica para evaluar la efectividad de los tests introduciendo pequeñas modificaciones, o mutaciones al código del programa determinando si esos cambios son detectados o no por los tests.

El **Predictive Mutation Testing (PMT)** usa Machine Learning para predecir el resultado de un mutante sin tener que ejecutar toda la suite de tests.

La efectividad de los modelos de PMT está muy influenciada por factores como el preprocesamiento de datos, el desbalanceo de clases y la selección de modelo. Estos datasets tienen un alto desbalanceo y sesga los modelos hacia la clase mayoritaria.

Este problema es el mismo que nos encontramos con los datasets de ciberseguridad, la idea de mi TFG sería usar técnicas de preprocesamiento y filtrado que se usa para PMT pero usarlo en los datasets de ciberseguridad.

Para intentar arreglar este desbalanceo, el paper usa técnicas como **RandomUnderSampler (RUS)**, **Synthetic Minority Over-sampling Technique (SMOTE)** y **Edited Nearest Neighbors (ENN)**.

MÉTODOS

Preprocesamiento de datos

Para hacer un filtrado de los datos, elimina los *unreached mutants* que son muestras triviales en las que claramente, al no ser probados por ningún test, pasan los tests correctamente.

Para el caso de los datasets de ciberseguridad, habría que buscar muestras triviales (como los *unreached mutants*) que podamos eliminar para reducir el tamaño de los datasets.

Lo siguiente que hace es dividir los datos en 80% de entrenamiento y 20% de test. Esta división se realiza por proyectos en lugar de por mutantes. Esto se hace para que en los tests no aparezcan mutantes de proyectos que están en el entrenamiento.

En el caso de los datasets de ciberseguridad, la división sería simplemente un 80% de entrenamiento y 20% de test. No tenemos el problema de los proyectos y mutantes.

Después, realiza una normalización **RobustScaler**. Este tipo de normalización usa la mediana y el rango intercuartílico en lugar de la media y varianza. Esto permite que la normalización sea más robusta y menos sensible a los extremos.

$$x' = \frac{x - \text{mediana}}{\text{IQR}}$$
$$\text{IQR} = Q_3 - Q_1$$

IMPORTANTE: La normalización de los datos se realiza únicamente con el cálculo de la mediana y el IQR a partir de los datos de entrenamiento.

En el caso de los datasets de ciberseguridad, la normalización se realizaría de la misma forma. Recalcar que para calcular la mediana y el rango intercuartílico, sólo debemos usar los datos de entrenamiento.

Desbalanceo de clases

Para abordar el problema del desbalanceo de clases, el paper usa 3 técnicas de rebalanceo:

- **RandomUnderSampler (RUS)**: Elimina instancias aleatoriamente de la clase mayoritaria produciendo una distribución de 50%.
- **Synthetic Minority Nearest Neighbours (SMOTE)**: Genera ejemplos sintéticos de la clase minoritaria interpolando entre clases existentes. Esto logra una distribución del 50% también.
- **Edited Nearest Neighbours (ENN)**: Elimina instancias ambiguas o mal etiquetadas de la clase mayoritaria. Esto no logra una distribución del 50% pero lo acerca.

En el caso de los datasets de ciberseguridad, podría probar las técnicas de RUS, SMOTE y ENN. También se me ocurre que se podría hacer un primer proceso de ENN y después otro proceso de RUS o SMOTE para terminar de realizar el balanceo. Serían en total 5 posibilidades:

- RUS.
- SMOTE.
- ENN.
- ENN + RUS.
- ENN + SMOTE.

Métricas de evaluación

Para la evaluación de los modelos usa **Precision, Recall, F1 Score, Matthews Correlation Coefficient (MCC)** y **Area Under the Receiver Operating Characteristic Curve (AUCROC)**.

$$\begin{aligned} \text{Precision} &= \frac{TP}{TP + FP} & \text{F1 Score} &= 2 \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \\ \text{Recall} &= \frac{TP}{TP + FN} & \text{MCC} &= \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \end{aligned}$$

Esto se ve mas a fondo en otro paper.